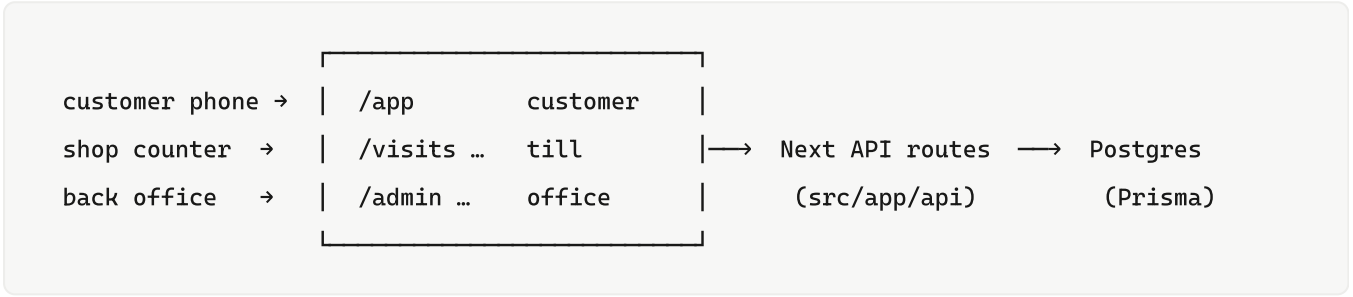
Design

How the system is put together, and why. Written to be read by whoever picks this up next — including future me.

Shape

One Next.js application, one Postgres database, three faces:



Everything shares one database and one set of rules. The rules themselves are plain functions in `src/lib`, with unit tests, and the API routes are thin wrappers over them. That is deliberate: a rule that lives in a screen gets copied and then diverges.

Module	Holds
`lib/booking.ts`	Free-slot maths, opening hours, shop-local time. Pure.
`lib/booking-server.ts`	Booking settings, hours, and finding free times in the database
`lib/booking-create.ts`	Saving an appointment — the one path both staff and customers use
`lib/payments.ts`	How a visit was paid: cash, card, InstaPay, or a mix. Pure.
`lib/finance.ts`	The money model: commission, expenses and advances by account
`lib/session-policy.ts`	How long a login lasts, per role. Pure.
`lib/roles.ts`	Who is staff, which paths belong to which door. Pure.

Roles

Role	Signs in with	Sees
`OWNER`	email	everything
`CASHIER`	email	the till, the diary, expenses, the queue
`BARBER`	email	their own day, their own customers, their own earnings
`CUSTOMER`	phone number	their own bookings and profile, nothing else

Three doors, enforced in `middleware.ts`: shop paths need a staff role, `/app` needs a customer, `/barber` needs a barber. The API enforces the same thing independently (`requireStaff`, `requireBarber`, `requireCustomer`), because middleware protects pages, not data.

A barber may mark someone arrived, started or a no-show, and nothing else: taking the money stays at the till, and they cannot move, cancel or re-price a booking. That is what keeps the diary and the takings from drifting apart.

> A path check on the prefix `"/app"` once let a customer open `/appointments`, the shop's own screen. Paths are matched segment by segment now (`isCustomerPath`), with tests — and the same care applies to `/barber` against the shop's `/barbers` screen.

The data model

The money side already existed; booking was added beside it.

```
Customer ──< Appointment >── Barber
                 │
                 ├──< AppointmentService   (name, price, length as quoted)
                 │
                 └─── Visit                (set when it is rung up)
                          │
                          ├──< VisitItem
                          └─── CashDrawerSession
```

An appointment holds a chair. A visit is money. They meet at `Appointment.visitId`, which is set once, when the customer pays. Until then the appointment carries only a quote. Ringing up is one

transaction: the visit is created and the appointment is stamped with it, so an appointment can never become two visits, and the shop can change the price at the till without rewriting what was quoted.

A customer who is served and leaves without paying is closed as `COMPLETED_UNPAID`: the chair was used, so the time stays taken, but no visit exists and no money is counted. If they come back and settle, it can still be rung up.

A walk-in is the same appointment, with `source = WALK_IN`, starting now, possibly with nobody assigned. That is what keeps one diary, one set of takings and one customer history however the customer arrived.

Decisions worth keeping:

- **Lines are snapshots.** `AppointmentService` copies the service name, price and length at booking time, as `VisitItem` already did. Raising the price of a haircut never rewrites last week's booking.
- **Cancelled frees the chair; completed does not.** One list of statuses (`BLOCKING_STATUSES`) decides this, used by the slot maths and the clash check alike.
- **"Any barber" is resolved at booking time**, not left vague: the customer gets a named barber immediately, so the shop can see who is doing what.
- **Hours are layered.** `ShopHours` is the week; `BarberWorkingHours` narrows it for one barber, never widens it. A barber with no rows simply works the shop's hours, so nothing has to be set up before booking works.
- **A barber with no `BarberService` rows does every service.** Same reason.

Time

The shop thinks in local time — "Saturday, 11:00 to 23:00" — and Egypt changes its clocks twice a year. So opening hours are stored as minutes from midnight with a weekday, and turned into real moments through the shop's timezone (`Africa/Cairo`, a setting). `shopTime()` resolves the offset twice so it stays right on the changeover day; the tests pin both a summer and a winter date.

Everything stored in the database is a UTC instant. Only the boundaries convert.

Free times

`availableSlots()` walks the working window on the shop's grid (every 15 minutes by default) and keeps a start time only if the whole service fits before closing without touching anything busy. Busy means appointments that still hold the chair, plus time off, each widened by the buffer.

The same function answers the customer app, the shop's booking modal and the reschedule screen. There is no second implementation to disagree with it.

Double booking

Free times can be stale by the time someone taps one, so the check that matters happens on save, inside a `Serializable` transaction: the clash query and the insert are one atomic step. The loser of a race gets 409 "That time has just been taken", which the screens show as a message rather than an error.

Money

Unchanged by the booking work, and worth restating because everything eventually lands here:

- Revenue is visit totals; tips belong to the barber and are tracked apart.
- Three accounts: **cash** (the drawer), **card** (Visa, less its commission), **InstaPay** (a bank transfer, no commission, never in the drawer). Expenses and advances come out of whichever account paid them — `outflowAccount()` decides, in one place.
- Advances are payroll, not expenses: they move money without reducing revenue.
- A tip that was not handed over in cash is paid to the barber out of the drawer, and recorded as a movement.

Sessions

Shop devices sit on a counter, so a staff login ends after 2 hours, or 30 minutes idle. A customer's phone is their own, so a customer login lasts as long as they keep using it (30 days idle). One policy module, `role` as the input; the token's own expiry carries it, so the middleware enforces it too.

Conventions

- Screens are client components; the API is the security boundary.
- Every route checks its own permission. None trusts the middleware.
- Anything that rewrites or removes money writes an `AuditLog` line, fire-and-forget.
- Both languages, always: a new string is added to `messages/en.ts` and `messages/ar.ts` together, and the type makes a missing Arabic key a compile error.
- New rules arrive as a tested function in `lib/` first, then a screen.